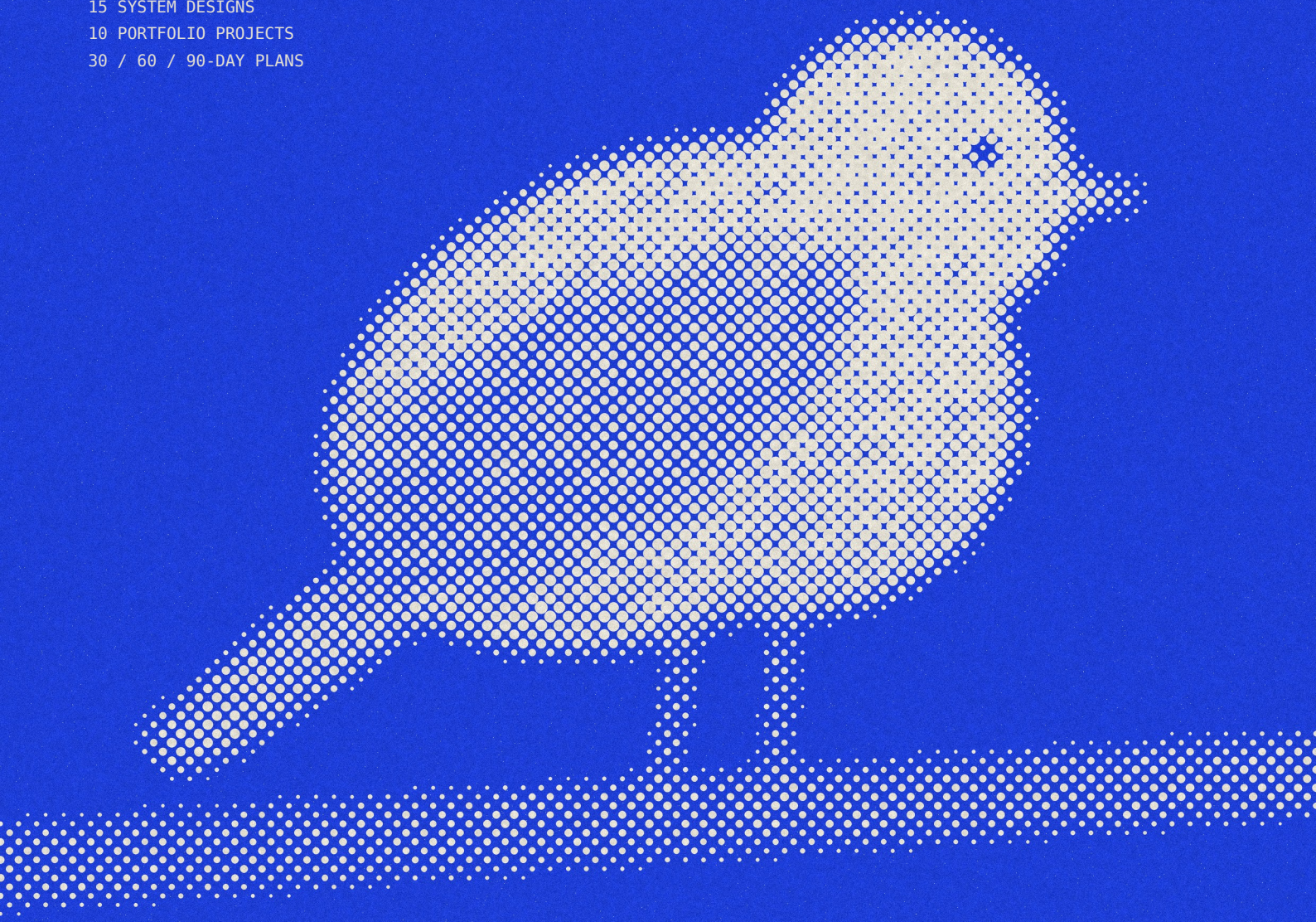
[INTERVIEW GUIDE 2027]

INFERENCE ENGINEERING INSIDER 2027

112 interview questions (36 candidate-reported), GPU systems, LLM inference and system design from leading AI companies.

20 JOB POSTINGS ANALYZED
15 SYSTEM DESIGNS
10 PORTFOLIO PROJECTS
30 / 60 / 90-DAY PLANS



Contents

[00] Read This First **8**
- What is inside 8
- Evidence tags 8
- Numbers to carry into every loop 9
- Three ways to use this guide 9

[1] Complete Inference Engineering Roadmap **10**
- What is inference engineering? 10
- Role responsibilities and career opportunities 10
- Inference vs ML vs MLOps vs AI infrastructure engineer 11
- Required foundations 11
- Beginner-to-advanced learning roadmap 12
- Essential mathematics and performance concepts 12

[2] Deep Learning, Transformer Architecture and LLM Internals **13**
- Neural network and transformer fundamentals 13
- Attention mechanisms: MHA, MQA and GQA 13
- Tokenization and autoregressive generation 14
- Prefill vs decode 14
- KV cache architecture 14
- Model architectures: dense, MoE and multimodal 15
- Computational complexity and memory requirements 15
- How LLM inference works internally 15

[3] GPU Architecture, CUDA and Performance Engineering **16**
- CPU vs GPU architecture 16
- NVIDIA data center GPUs: H100, H200, B200 and GB200 16
- CUDA programming and kernels 16
- GPU memory hierarchy and bandwidth 17
- Tensor cores and parallel execution 17
- Triton programming 17
- FlashAttention and kernel fusion 18
- Nsight Systems and Nsight Compute 18
- Compute-bound vs memory-bound workloads 18
- Profiling and performance optimization workflow 19

[4] LLM Inference Optimization and Model Serving **20**
- vLLM, SGLang, TensorRT-LLM and NVIDIA Triton 20
- Continuous batching and dynamic batching 20
- PagedAttention 21
- Prefix caching and KV cache optimization 21
- Speculative decoding 21
- Chunked prefill 21
- Quantization: FP16, BF16, FP8, INT8 and INT4 22
- Tensor and pipeline parallelism 22
- Disaggregated prefill and decode 22
- Throughput, latency, TTFT, TPOT and cost 22

[5] Distributed Inference and AI Infrastructure **24**
- Multi-GPU and multi-node inference 24
- NCCL, NVLink, NVSwitch and InfiniBand 24
- RDMA and GPU communication 24
- Expert parallelism and MoE serving 25

Distributed KV cache . . . 25
Kubernetes and Docker . . . 25
Ray Serve and distributed scheduling . . . 25
Load balancing and autoscaling . . . 25
Fault tolerance and high availability . . . 26
Production observability: Prometheus, Grafana and OpenTelemetry . . . 26

[6] Real Job Market Research and Technology Stack Analysis . . . 27
The 20 postings . . . 27
Company to skills to tools map . . . 31
Most requested technologies, ranked . . . 31
Junior, senior, staff and principal compared . . . 32

[7] 112 Insider Inference Engineering Interview Questions and Answers . . . 33
7.1 Inference Engineering Fundamentals . . . 33
Q001 What is your experience with inference optimization for LLMs? . . . 33
Q002 Why is LLM inference memory-bound? . . . 34
Q003 What are TTFT, TPOT and throughput, and how do they trade against each other? . . . 34
Q004 Walk me through how a transformer forward pass actually executes on a GPU. . . . 35
Q005 Estimate the maximum decode throughput of a 70B model on one 8xH100 node from first principles. 36
Q006 What is goodput, and why is it a better target than throughput? . . . 36
Q007 How do you approach cost optimization for an AI system that is exceeding its budget? . . . 37
Q008 Synchronous vs asynchronous inference: when do you use each? . . . 37
7.2 Transformer and LLM Internals . . . 38
Q009 What is a KV cache? What is FlashAttention? (asked back to back) . . . 38
Q010 Explain what is cached per layer in a KV cache and the correctness pitfalls when adding one (masking, positions, RoPE, shapes). . . . 38
Q011 Explain prefill vs decode and why serving systems treat them differently. . . . 39
Q012 Compare MHA, MQA and GQA. How does the choice affect KV memory and tensor parallelism? . . . 40
Q013 Compute the KV cache for Llama-3 70B at 32k context and batch 16. Does it fit on 8 H100s? 40
Q014 How does RoPE work, and what breaks when you extend a model's context window? . . . 41
Q015 Dense vs mixture-of-experts: what changes for serving? . . . 41
Q016 Why can sampling become a bottleneck, and how do you implement top-p efficiently on a GPU? 42
7.3 GPU Architecture and CUDA . . . 42
Q017 Explain CUDA's execution model, memory hierarchy and occupancy, then optimize a GEMM with tiling and coalescing. . . . 42
Q018 Explain Amdahl's law and how it applies to GPU matrix multiplication and end-to-end speedups. 43
Q019 How would you compute a matrix multiplication efficiently on a GPU without using shared memory? . . . 43
Q020 Walk me through a CUDA kernel you wrote: the optimizations, the challenges and exactly what you did. . . . 44
Q021 What is warp divergence, and how do you avoid it? . . . 45
Q022 Compare H100, H200 and B200 for serving a 70B dense model. . . . 45
Q023 What are tensor cores, and what must be true to keep them fully used? . . . 46
Q024 What are shared memory bank conflicts, and how do you fix the classic transpose? . . . 46
7.4 GPU Performance Optimization . . . 47
Q025 A kernel reaches 30% of peak memory bandwidth on an H100. How do you find out why? . . . 47
Q026 Your reduction is slower at 64 threads per block than at 256. What is going on? . . . 47
Q027 Give an example of a kernel that reached high tensor core utilization. How did you accomplish it? . . . 48
Q028 Design and benchmark an optimized inference pipeline: TorchDynamo, TorchInductor, external backends, fusion, CUDA graphs, quantization. . . . 49
Q029 Do a roofline analysis: is a decode GEMM at batch 16 for an 8k by 8k BF16 weight memory- or compute-bound on H100? . . . 49

Q030 What are CUDA graphs, and when do they help LLM inference? . . . 50
Q031 nvidia-smi shows 100% GPU utilization but throughput is low. Explain. . . . 50
Q032 How do you use Nsight Systems and Nsight Compute together in a performance investigation? 51
7.5 CUDA Kernels and Triton . . . 51
Q033 Implement a high-performance FP32 CUDA kernel for $C = A \cdot B$ (A is $m \times k$, B is $k \times n$), and explain your architecture choices. . . . 51
Q034 Design a deterministic CUDA reduction that guarantees a well-defined reduction order. (Hint given: similar to prefix sum.) . . . 52
Q035 How do TensorRT-LLM and vLLM work? (asked alongside "implement an inference loop in PyTorch") 52
Q036 Write a fused, numerically stable softmax kernel for rows of length up to 8,192. . . . 53
Q037 Derive the online softmax and explain how FlashAttention uses it. . . . 54
Q038 Write a Triton kernel for fused RMSNorm. When do you choose Triton over CUDA C++? . . . 54
Q039 Why does decode attention need split-KV (Flash-Decoding), and how does it work? . . . 55
Q040 How would you write a W4A16 dequantize-and-GEMM kernel for decode? . . . 55
7.6 KV Cache and Memory Management . . . 56
Q041 Design and implement key-value caching for autoregressive inference in a Transformer decoder: tensor shapes for prefill and decode, per-layer cache API, batching and beam search, mixed precision. . . . 56
Q042 Your LLM inference is running out of GPU memory with long conversations. What do you do? . 57
Q043 A chat assistant re-sends the whole conversation every turn. How do you avoid recomputing it, and what are the pitfalls? . . . 57
Q044 How does vLLM's PagedAttention work, and why does it matter for GPU memory efficiency? . . 58
Q045 KV memory is full. Do you swap or recompute preempted sequences? . . . 59
Q046 Design an eviction policy for a prefix cache shared by many tenants. . . . 59
Q047 What are the risks of an FP8 KV cache, and how do you validate it? . . . 60
Q048 How many concurrent 4k-token conversations can one 8xH100 node hold for Llama-3 70B in FP8 weights and BF16 KV? . . . 60
7.7 LLM Serving and Continuous Batching . . . 61
Q049 Design an inference batching system for a single GPU that handles up to 100 inputs per batch while users wait synchronously, maximizing utilization under compute constraints. . . . 61
Q050 Here is a junior developer's design for an inference batching system. Critique it and propose improvements. . . . 61
Q051 When does TensorRT-LLM win over vLLM? . . . 62
Q052 Design the scheduler for a continuous-batching inference engine. . . . 63
Q053 Batch requests with very different prompt and output lengths without head-of-line blocking. 63
Q054 We need to optimize inference at batch size 128. Should we use speculative decoding? . . 64
Q055 What problem does chunked prefill solve, and what is its main tuning knob? . . . 64
Q056 Choose an engine for three workloads: an agent platform with long shared prompts, a 405B FP8 API on H100s, and fast bring-up of a new open model. . . . 65
7.8 Quantization and Model Compression . . . 66
Q057 What is quantization? What are QAT and QAD? . . . 66
Q058 Describe common AI optimization techniques for training and inference: quantization, pruning, distillation, kernel fusion, memory and throughput optimizations. . . . 66
Q059 Explain quantization and model distillation for inference optimization: how do you choose between them? . . . 67
Q060 FP8 E4M3 vs E5M2: which do you use for inference, and how are scales chosen? . . . 67
Q061 Compare AWQ, GPTQ and SmoothQuant. . . . 68
Q062 Why can W4A16 speed up decode but slow down prefill at large batch? . . . 68
Q063 How do you validate a quantized model's quality before rollout? . . . 69
Q064 What are NVFP4 and MXFP4, and why do they need block scaling? . . . 69
7.9 Distributed Inference . . . 70
Q065 Explain tensor parallelism versus pipeline parallelism, and when you would use each. . . . 70
Q066 Design multi-GPU matrix multiplication. . . . 71

Q067 Why would you disaggregate prefill and decode onto separate machines, as Mooncake does? What does it buy and what does it cost? . . . 71
Q068 On a GPU you batch to amortize weight reads. What is the batching calculus on an SRAM-only machine, and how should that change pricing? . . . 72
Q069 How many GPUs do you need to serve Llama 3.1 405B, and which parallelism do you use? . . . 72
Q070 Explain expert parallelism and how you handle MoE load imbalance. . . . 73
Q071 How would you prefill a 1M-token prompt? . . . 74
Q072 You have 16 H100s in two nodes for a 70B model. TP=16, or two TP=8 replicas? . . . 74
7.10 Networking and GPU Communication . . . 74
Q073 Explain your experience, then questions around GPUs, CI/CD, inter-GPU communication, Kubernetes and NVIDIA plugins for Kubernetes. . . . 74
Q074 Ring vs tree all-reduce: costs and when each is used. . . . 75
Q075 NVLink, PCIe and InfiniBand: when does each become the bottleneck in inference? . . . 76
Q076 How does GPUDirect RDMA work, and why does disaggregated serving depend on it? . . . 76
Q077 An NCCL all-reduce hangs or times out in production. How do you debug it? . . . 77
Q078 Why do inference engines write their own all-reduce instead of using NCCL for decode? . . . 77
Q079 What does the all-to-all in MoE expert parallelism cost, and how do you hide it? . . . 78
Q080 How should you place a TP=8 replica in a rail-optimized GPU cluster, and why does topology matter? . . . 78
7.11 Inference System Design . . . 79
Q081 Design a high-concurrency LLM inference service: GPU utilization, KV cache memory, batching and scheduling, request splitting and merging. . . . 79
Q082 Design a GPU-backed inference API for serving multiple ML models: SLOs, dynamic batching, versioning and A/B routing, autoscaling across heterogeneous GPU nodes, backpressure, multi-tenant isolation, model loading and warmup. . . . 80
Q083 Design a system for distributing large model weight files (tens to hundreds of GB, sharded) to a fleet of inference workers. . . . 80
Q084 Design an API that lets users sample from large language models efficiently, with good batching and request orchestration. . . . 81
Q085 Design a distributed rate limiter for an LLM inference API. . . . 81
Q086 Coding: manage GPU credits. Then design a scalable AI inference service. (60 + 60 minute phone screen) . . . 82
Q087 Design an inference service that hosts about 200 open models on shared GPUs, including how to decide which models stay loaded and which get evicted. . . . 83
Q088 Support multi-tenant LoRA serving so one customer’s adapter cannot starve another’s. . . . 83
7.12 Production Reliability and Observability . . . 84
Q089 How would you troubleshoot a Kubernetes pod that is stuck in Pending status? . . . 84
Q090 Explain the LLM inference stack and its failure scenarios. . . . 84
Q091 Build a dedicated endpoint with strict tail-latency guarantees, and explain what differs from the shared pool. . . . 85
Q092 Determinism is the headline claim. What does it actually buy at p99, and why does it matter for agentic workloads? . . . 86
Q093 p99 TTFT doubled right after a deploy. Walk through the investigation. . . . 86
Q094 What metrics and alerts would you put on an LLM inference service? . . . 87
Q095 How do you safely roll out a new attention kernel or quantized model to production? . . . 87
Q096 One GPU in a TP=8 replica starts throwing XID errors. What happens and what should your system do? . . . 88
7.13 Coding and Performance Debugging . . . 88
Q097 Implement an inference loop in PyTorch. . . . 88
Q098 Implement topological sort in Python. . . . 89
Q099 Serialize and deserialize a binary tree, with clean code and optimal time and space. . . . 90

Q100 Implement speculative decoding: one normal decoding function and one speculative function speculative_decoding(model, tokenizer, prompt, draft, max_tokens) that must run faster than normal decoding. . . . 90
Q101 Take-home: build a simplistic version of an AI chat playground (a ChatGPT-like interface) using the company's models. . . . 91
Q102 You turned on speculative decoding and throughput went down. What happened? . . . 92
Q103 Given request arrival times and durations, find the peak number of concurrent requests on one GPU. . . . 92
Q104 Write a simulator for a continuous-batching scheduler with a token budget, and report TTFT and TPOT. . . . 93
7.14 Advanced Inference Architecture . . . 94
Q105 Design a high-concurrency HTTP API for language-model inference where GPU workers process compatible requests in dynamic batches on a limited GPU pool. . . . 94
Q106 Design a service that accepts low-latency online inference requests, batches compatible requests, routes work to GPU workers and supports multiple models and versions concurrently. . . . 94
Q107 Design a file uploader for an AI chat app that handles large multimodal uploads and feeds them into a model's context. . . . 95
Q108 Design a batched inference system where 100 requests take the same time as 1. . . . 95
Q109 Serve structured outputs (JSON schema, grammars) at scale without hurting throughput. . . . 96
Q110 Design inference for reinforcement learning rollouts. . . . 97
Q111 Reasoning models generate thousands of thinking tokens. What changes in serving? . . . 97
Q112 Serve the same model across NVIDIA, AMD and TPU fleets. What has to be true? . . . 98

[8] Advanced Inference System Design . . . 99

8.1 LLM inference service for one million users . . . 99
8.2 ChatGPT-style inference backend . . . 100
8.3 Multi-tenant GPU inference platform . . . 101
8.4 High-throughput vLLM cluster . . . 102
8.5 Distributed KV cache . . . 103
8.6 Inference for a 405B-parameter model . . . 104
8.7 Multi-GPU inference on one node . . . 105
8.8 Low-latency inference API . . . 106
8.9 Autoscaling GPU cluster . . . 107
8.10 MoE inference platform . . . 108
8.11 Speculative decoding infrastructure . . . 109
8.12 Multi-region inference . . . 110
8.13 Fault-tolerant inference service . . . 111
8.14 Platform optimized for cost per million tokens . . . 112
8.15 Real-time multimodal inference service . . . 113

[9] Hands-On Engineering Projects and Portfolio . . . 115

P1 Build an LLM inference service using vLLM . . . 115
P2 Benchmark vLLM vs SGLang vs TensorRT-LLM . . . 116
P3 Implement continuous batching from scratch . . . 116
P4 Benchmark PagedAttention . . . 116
P5 Build a GPU profiling workflow . . . 117
P6 Implement quantized model serving . . . 117
P7 Build multi-GPU inference . . . 117
P8 Experiment with speculative decoding . . . 118
P9 Deploy inference infrastructure using Kubernetes . . . 118
P10 Build an inference observability platform . . . 118
Turning projects into a portfolio . . . 119

[10] Company-Specific Interview Preparation and 30/60/90-Day Roadmap . . . 120

OpenAI . . . 120
Anthropic . . . 120
NVIDIA . . . 121
Google DeepMind . . . 121
Meta . . . 122
xAI . . . 122
Together AI . . . 122
Fireworks AI . . . 123
Baseten . . . 123
CoreWeave . . . 123
30-day intensive track . . . 123
60-day intermediate track . . . 124
90-day advanced track . . . 124
Core technical resources . . . 125
Inference Engineering Interview Readiness Checklist . . . 125

[11] Sources . . . 127
Question sources: candidate reports [R] . . . 127
Question sources: vendor and public banks [V] . . . 128
Job postings (Chapter 6) . . . 128
Interview process and context . . . 130
Hardware and technical references . . . 130
Join us . . . 130

READ THIS FIRST

[START HERE]

Inference engineering is the job of turning a trained model into a fast, cheap and reliable service. The people who do it sit between research and production: they read attention kernels, size GPU fleets, tune schedulers and carry the pager when p99 latency doubles at 2 a.m. This guide prepares you for the interviews that hire them.

» WHAT IS INSIDE

Chapters 1-5 The knowledge base: the role, transformer internals, GPU architecture and CUDA, serving and optimization, and distributed inference. Every chapter ends with the numbers you should be able to say without notes.

Chapter 6 A job market study of 20 real postings from 16 companies, with the original URLs, a company-to-skills-to-tools map, a ranked list of the most requested technologies and a comparison by seniority.

Chapter 7 112 interview questions in 14 categories. Every card names its source, its stage when known, an evidence tag and a difficulty, then answers in four parts: what the interviewer tests, the answer, what a senior answer adds, and the traps. Two follow-ups close each card, answered.

Chapter 8 15 system design scenarios worked end to end with capacity math, an ASCII architecture diagram, GPU allocation, failure handling, monitoring, cost and trade-offs.

Chapter 9 10 portfolio projects with architecture, steps, a benchmarking method and what to publish. We give no performance results: you measure your own.

Chapter 10 Ten company profiles, three study plans (30, 60 and 90 days) and the readiness checklist.

» EVIDENCE TAGS

Every question card carries one tag. We separate what candidates reported from what we built, because a prep guide that blurs the two wastes your time.

R Reported A candidate report names the company: a Glassdoor review, a PracHub listing submitted by a candidate (company, round and date shown), a Taro or Aced (formerly Exponent) experience, a 1Point3Acres thread or a LeetCode discussion post. Wording is often paraphrased by the platform. Treat the topic as the signal.

V Vendor A prep vendor or a public question bank attributes the question to a company or says candidates "report" it, but we could not see the underlying candidate report.

P Practice An industry-standard preparation question that we wrote or reconstructed from job postings and the topics reported in loops. No company is claimed.

[HOW TO READ A SOURCE LINE]

A card that says *NVIDIA // Technical screen // Hard* with a Glassdoor link means one candidate reported that topic in that round. It does not mean every NVIDIA loop asks it. Loops change by team, level and quarter. Confirm the format with your recruiter.

» NUMBERS TO CARRY INTO EVERY LOOP

[HARDWARE AND MODEL CONSTANTS USED IN THIS GUIDE]

GPU	HBM	BANDWIDTH	BF16 DENSE	FP8 DENSE	NVLINK (bidir)
H100 SXM	80 GB	3.35 TB/s	~989 TFLOPS	~1979 TFLOPS	900 GB/s
H200	141 GB	4.8 TB/s	~989 TFLOPS	~1979 TFLOPS	900 GB/s
B200	192 GB*	8.0 TB/s	~2250 TFLOPS	~4500 TFLOPS	1.8 TB/s
GB200 (NVL72)	186 GB	8.0 TB/s	~2500 TFLOPS	~5000 TFLOPS	1.8 TB/s

* about 180 GB usable per GPU in an 8-GPU HGX B200 system

MODEL (GQA)	LAYERS	KV HEADS	HEAD DIM	KV BYTES/TOKEN (BF16)
Llama-3 8B	32	8	128	128 KiB
Llama-3 70B	80	8	128	320 KiB
Llama-3.1 405B	126	8	128	504 KiB

RULES OF THUMB

weights (bytes)	= params x bytes per param
KV/token (bytes)	= 2 x layers x kv_heads x head_dim x bytes
decode FLOPs/token	~ 2 x params (dense)
decode floor/token	~ weight bytes / aggregate HBM bandwidth
H100 ridge point	~ 989e12 / 3.35e12 ~ 295 FLOP per byte (BF16)

Vendor figures are dense (no sparsity) and rounded; check the datasheet for the SKU you are discussing. The KV figures follow from each model's published config. Meta's own inference team quotes the same 320 KiB per token for Llama-3 70B in its QCon talk on LLM serving.

» THREE WAYS TO USE THIS GUIDE

[1] **Two weeks out:** read Chapters 2 to 5, then answer the Chapter 7 cards out loud with the answer panel covered.

[2] **One week out:** run three Chapter 8 designs on a whiteboard in 45 minutes each, then build one Chapter 9 project to talk about.

[3] **Day before:** reread the numbers above, the company profile in Chapter 10 and the checklist on the last pages.

[SECTION 01 / 11]

COMPLETE INFERENCE ENGINEERING ROADMAP

==========

[THE ROLE, THE MAP AND THE MATH]

What the job is, how it differs from its neighbours, what you need to know first and the order in which to learn the rest.

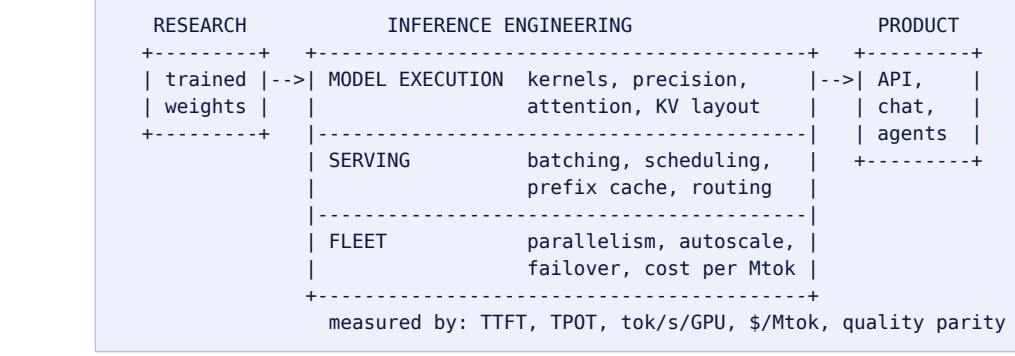
> [AT A GLANCE]
> Inference engineers own the path from a trained checkpoint to tokens on a user's screen. The work splits into three layers: the **model execution** layer (kernels, precision, attention, KV cache), the **serving** layer (batching, scheduling, caching, routing) and the **fleet** layer (parallelism, autoscaling, reliability, cost). Interviews test all three, and the strongest candidates move between them in one answer.

» WHAT IS INFERENCE ENGINEERING?

Training produces weights. Inference engineering makes those weights earn money. A frontier model can cost more to serve over its life than it cost to train, because every user request runs it again. The postings in Chapter 6 describe the job in the same terms: Anthropic's inference team says it focuses on scaling inference systems, reliability, compute efficiency and new inference capabilities; DeepMind's staff inference performance role asks for engineers who raise throughput per GPU and cut latency across the application, model and fleet layers.

The job has one objective with three constraints. The objective is **useful tokens per dollar**. The constraints are latency (time to first token and time per output token), quality (the served model must match the reference model) and reliability (the service keeps its SLO through failures and launches).

[FIG 1.1 // WHERE THE INFERENCE ENGINEER WORKS]



» ROLE RESPONSIBILITIES AND CAREER OPPORTUNITIES

Across the postings we studied, the work falls into seven recurring responsibilities:

> **Performance:** profile and remove bottlenecks across kernels, runtime and scheduler (NVIDIA TensorRT-LLM, Baseten Model Performance, DeepMind).

> **Serving systems:** build batching, caching and request routing (Anthropic, Microsoft, Perplexity).

> **New model bring-up:** support new architectures, MoE, multimodal and new hardware (OpenAI AMD enablement, AWS Neuron, Cerebras).
> **Fleet efficiency:** autoscaling, capacity planning, bin packing (Anthropic launch engineering, CoreWeave).
> **Correctness:** numerics, quantization quality, regression gates (Anthropic performance engineer, xAI RL inference).
> **Reliability:** on-call, deployment pipelines, benchmarking in CI (xAI applied inference, OpenAI inference runtime productivity).
> **Open source:** vLLM, SGLang and TensorRT-LLM contributions are asked for by name at xAI, NVIDIA, Meta, CoreWeave and Google.

Career paths branch three ways. The **kernel and performance** path leads to roles like NVIDIA's DL algorithms engineer or Anthropic's GPU performance engineer. The **serving systems** path leads to inference platform and runtime teams. The **fleet and platform** path leads to capacity, launch and cloud inference teams. Staff and principal roles expect you to own one path and speak fluently about the other two.

» INFERENCE VS ML VS MLOPS VS AI INFRASTRUCTURE ENGINEER

[TABLE 1.1 // FOUR NEIGHBOURING ROLES]

	INFERENCE ENG	ML ENGINEER	MLOPS ENGINEER	AI INFRA ENG
CORE OUTPUT	fast, cheap, correct serving	a better model or feature	reliable pipelines train to deploy	GPU clusters, network, storage
MAIN METRIC	tok/s/GPU, TTFT, TPOT, $/Mtok	offline/online model quality	deploy freq, drift, MTTR	utilization, uptime, cost
DAILY TOOLS	vLLM, SGLang, TRT-LLM, CUDA, Nsight, Triton	PyTorch, data, eval harness	MLflow, K8s, CI/CD, registry	K8s, Slurm, NCCL, IB, RDMA
LANGUAGES	Python, C++, CUDA, Rust	Python, SQL	Python, Go, Terraform	Go, Python, C++, Bash
INTERVIEW SIGNATURE	KV math, kernels, batching design	ML theory, model design, coding	pipelines, infra design, debugging	distributed systems, network

The overlap is real. An ML engineer who has tuned vLLM is halfway to inference. An AI infrastructure engineer who has debugged NCCL hangs already owns the fleet layer. The gap to close is almost always the middle: the scheduler, the KV cache and the arithmetic of memory bandwidth.

» REQUIRED FOUNDATIONS

Python Async I/O, multiprocessing, profiling (py-spy, cProfile), PyTorch internals at the level of tensors, dtypes, streams and torch.compile.

C++ Modern C++ (RAII, move semantics, templates), memory layout, threading. Runtimes and kernels at NVIDIA, Microsoft and Groq are C++.

Linux and OS Processes vs threads, virtual memory and paging (PagedAttention borrows the idea), NUMA, huge pages, cgroups, perf, strace.

Architecture Caches, memory bandwidth vs latency, SIMD and SIMT, PCIe vs NVLink, why a GPU hides latency with parallelism.

Networking TCP vs RDMA, bandwidth-delay product, HTTP/2 and gRPC streaming, server-sent events, load balancer behaviour with long-lived streams.

Dist. systems Queues and backpressure, consistent hashing, leader election, idempotency, tail latency, failure domains.

» BEGINNER-TO-ADVANCED LEARNING ROADMAP

[FIG 1.2 // SIX STAGES, EACH WITH AN EXIT TEST]

```
STAGE 1  FOUNDATIONS        Python, C++, Linux, one systems course
         exit test: explain virtual memory and write a thread pool
STAGE 2  TRANSFORMERS       attention, MHA/MQA/GQA, RoPE, sampling
         exit test: write a GPT forward pass with a KV cache from scratch
STAGE 3  GPU BASICS         SMs, warps, HBM vs SRAM, roofline, CUDA 101
         exit test: tiled GEMM and a reduction, profiled in Nsight Compute
STAGE 4  SERVING            vLLM/SGLang, continuous batching, paging,
                            prefix cache, quantization, speculative decode
         exit test: serve a 8B model and explain every number in the bench
STAGE 5  DISTRIBUTED        TP, PP, EP, NCCL, disaggregated prefill/decode
         exit test: size a 405B deployment on paper and on 8 GPUs
STAGE 6  PRODUCTION         SLOs, autoscaling, multi-tenant, cost per Mtok
         exit test: design a multi-region service and defend its failure modes
```

» ESSENTIAL MATHEMATICS AND PERFORMANCE CONCEPTS

Intensity FLOPs per byte moved. A matrix-vector product (decode at batch 1) does about 2 FLOPs per weight read, so it sits far below the H100 BF16 ridge of about 295 FLOP per byte. Decode is memory-bound until the batch is large.

Roofline Attainable FLOP/s $= \min(\text{peak FLOP/s},\ I \times \text{bandwidth})$. Every kernel discussion should start by placing the kernel on this line.

Little's law $L = \lambda W$. Concurrent sequences equal arrival rate times time in system. At 50 requests per second and 8 seconds per request you hold 400 sequences, which sets the KV budget.

Amdahl's law Speedup $= 1/((1-p)+p/s)$. If 20% of step time is host overhead, a perfect kernel caps you at 5x. This is why CUDA graphs matter.

Percentiles Report p50, p95, p99 and p99.9. Averages hide queueing. A request that waits behind a long prefill shows up only in the tail.

Queueing As utilization $\rho \to 1$, wait time grows like $\rho/(1-\rho)$. Running GPUs at 95% busy trades throughput for tail latency.

Numerics FP16 vs BF16 range, FP8 E4M3 vs E5M2, accumulation in FP32, why softmax needs the max-subtraction trick.

[NUMBERS TO KNOW]

Two FLOPs per parameter per token for a dense forward pass. 320 KiB of KV per token for Llama-3 70B in BF16. 3.35 TB/s for H100, 4.8 TB/s for H200, 8 TB/s for B200. Ridge point near 300 FLOP per byte. Little's law to turn traffic into concurrency, and concurrency into KV memory.

[SECTION 02 / 11]

DEEP LEARNING, TRANSFORMER ARCHITECTURE AND LLM INTERNALS

[WHAT ACTUALLY RUNS ON THE GPU]

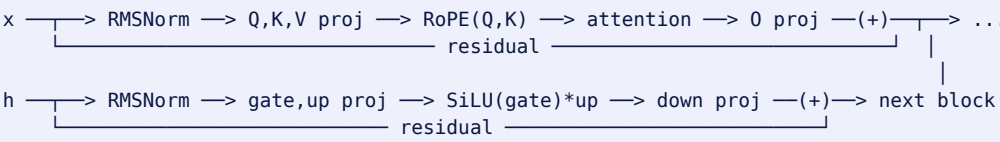
From tokens to logits, with the memory and FLOP counts that decide every serving choice.

> [AT A GLANCE]
> A decoder-only transformer turns a sequence of token IDs into a probability over the next token. Inference runs it in two phases: **prefill** processes the whole prompt in parallel and is compute-bound; **decode** produces one token per step per sequence and is memory-bound. The **KV cache** is what makes decode cheap in FLOPs and expensive in memory. Almost every optimization in this book attacks one of those two facts.

» NEURAL NETWORK AND TRANSFORMER FUNDAMENTALS

A transformer block has two sublayers, each wrapped in a residual connection with normalization (RMSNorm in most current open models):

[FIG 2.1 // ONE DECODER BLOCK (PRE-NORM)]



For hidden size d and MLP width d_{ff}, the attention projections hold about $4d^2$ parameters (less with GQA) and a SwiGLU MLP holds $3d\,d_{ff}$. With L layers the parameter count is roughly $L(4d^2 + 3d\,d_{ff})$ plus embeddings. A dense forward pass costs about $2N$ FLOPs per token for N parameters, plus the attention term that grows with context.

» ATTENTION MECHANISMS: MHA, MQA AND GQA

Scaled dot-product attention for one head:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}} + M\right)V$$

where M is the causal mask. The variants differ in how many key/value heads they keep:

MHA	H query heads, H KV heads. Best quality per parameter, largest KV cache.
MQA	H query heads share 1 KV head. KV shrinks by Hx. Quality can drop and tensor parallelism gets awkward because one KV head cannot be split across GPUs.
GQA	H query heads share G KV heads ($1 < G < H$). Llama-3 uses 64 query and 8 KV heads at 70B: an 8x KV saving with near-MHA quality. TP up to G splits cleanly.
MLA	DeepSeek's multi-head latent attention caches a compressed latent per token and up-projects at use time. Smaller cache, more compute per step, and kernels must be written for it.

» TOKENIZATION AND AUTOREGRESSIVE GENERATION

Production LLMs use byte-level BPE or SentencePiece vocabularies of roughly 100k to 260k entries. Tokenization runs on the CPU and can become a bottleneck at high QPS; fast Rust tokenizers and caching of system-prompt token IDs fix most of it. Generation is a loop:

[LISTING 2.1 // THE GENERATION LOOP WITH A KV CACHE]

```
def generate(model, ids, max_new, temperature=1.0):
    cache = model.new_cache()
    logits = model(ids, cache=cache)           # PREFILL: all prompt tokens at once
    out = []
    for _ in range(max_new):                   # DECODE: one token per step
        nxt = sample(logits[:, -1] / temperature)
        out.append(nxt)
        if nxt == EOS: break
        logits = model(nxt[:, None], cache=cache)   # reads all past K,V; appends one
    return out
```

Sampling (temperature, top-k, top-p, min-p, repetition penalties, structured output masks) runs per step on the logits. With a 128k vocabulary and large batches, a naive sort for top-p shows up in profiles; serving engines use fused sampling kernels.

» PREFILL VS DECODE

[TABLE 2.1 // THE TWO PHASES]

	PREFILL	DECODE
tokens per step	whole prompt (P tokens)	1 per sequence
main op	GEMM [P x d] x [d x d]	GEMV-like [B x d] x [d x d]
intensity	high (~P FLOP per weight byte)	low (~B FLOP per weight byte)
bound by	compute (tensor cores)	HBM bandwidth (weights + KV)
latency metric	TTFT (time to first token)	TPOT / ITL (time per output token)
scales with	prompt length, quadratic attention	batch size, context length

Because decode reads every weight once per step regardless of batch, adding sequences to a decode batch is almost free until the batch is large enough to cross the ridge point. That one fact is the reason continuous batching exists.

» KV CACHE ARCHITECTURE

Each layer stores K and V for every past token: $2 \times L \times G \times d_h \times$ bytes per token. For Llama-3 70B in BF16 that is $2 \times 80 \times 8 \times 128 \times 2 = 327{,}680$ bytes, or 320 KiB per token. A single 32k-token conversation therefore holds 10 GiB of cache. A batch of 64 such conversations needs 640 GiB, more than eight H100s hold in total. The KV cache, not the weights, is the scarce resource at long context.

Serving engines manage it like an operating system manages RAM:

> **Paged allocation** (vLLM PagedAttention): fixed-size blocks, a block table per sequence, near-zero fragmentation.

> **Prefix sharing**: blocks for identical prefixes (system prompts, few-shot examples, earlier turns) are shared copy-on-write. SGLang's RadixAttention keeps them in a radix tree.

> **Tiering**: cold blocks spill to CPU memory or SSD and come back on a cache hit, trading PCIe bandwidth for recompute.

> **Compression**: FP8 or INT8 KV halves memory; sliding-window and attention-sink schemes cap growth for some models.

» MODEL ARCHITECTURES: DENSE, MOE AND MULTIMODAL

Dense Every token uses every weight. Simple to shard with TP; decode cost tracks total parameters.

MoE A router sends each token to k of E experts per layer. Compute per token tracks *active* parameters, memory tracks *total* parameters. Serving needs expert parallelism, all-to-all communication and load balancing across experts. Small batches touch many experts and lose the compute advantage, so MoE serving favours large batches.

Multimodal A vision or audio encoder turns pixels or audio frames into embeddings that join the token stream. Images add hundreds to thousands of tokens of prefill each; encoders can run on separate GPUs and their outputs can be cached by content hash.

» COMPUTATIONAL COMPLEXITY AND MEMORY REQUIREMENTS

[FIG 2.2 // SIZING A DEPLOYMENT ON ONE PAGE]

```
weights      = N x bytes/param               70B BF16 = 140 GB ; FP8 = 70 GB
KV per token = 2 x L x G x d_h x bytes       70B BF16 = 320 KiB
KV total     = KV/token x tokens in flight   400 seqs x 4k tok = 1.6M tok ~ 490 GiB
prefill      ~ 2 x N x P  FLOPs (+ attention ~ 4 x L x P^2 x d)
decode step  ~ 2 x N x B  FLOPs, reads weights once + all KV of the batch
decode floor ~ (weight bytes + KV bytes read) / aggregate HBM bandwidth
example      70B FP8 on 4 x H100: 70 GB / 13.4 TB/s ~ 5.2 ms per step before KV
```

Attention FLOPs grow with P^2 during prefill and with context length per decode step. At 128k context, attention dominates the MLP in prefill and the KV read dominates the weight read in decode, which is why long-context serving is a different engineering problem from chat.

» HOW LLM INFERENCE WORKS INTERNALLY

One engine iteration in a modern server (vLLM V1, SGLang, TensorRT-LLM) looks like this:

[1] **Schedule:** pick which running sequences decode, which waiting requests prefill (possibly in chunks), and whether anything must be preempted to free KV blocks.

[2] **Prepare inputs:** build flattened token IDs, positions and block tables on the GPU. Host overhead here is a classic bottleneck at small batch.

[3] **Forward:** run the model, usually through CUDA graphs captured per batch size, with fused attention kernels (FlashAttention, FlashInfer) that read paged KV.

[4] **Sample:** apply logits processors, grammar masks and the sampler.

[5] **Detokenize and stream:** incremental detokenization on the CPU, then SSE or gRPC streaming to the client.

[6] **Update state:** append KV, free blocks of finished sequences, update prefix cache metadata.

[NUMBERS TO KNOW]

Prefill is compute-bound, decode is memory-bound. KV per token $= 2LGd_h \times$bytes. 70B in BF16 is 140 GB of weights and 320 KiB of KV per token. MoE compute tracks active parameters while memory tracks total parameters.

[SECTION 03 / 11]

GPU ARCHITECTURE, CUDA AND PERFORMANCE ENGINEERING

[MAKE THE HARDWARE DO THE WORK]

The memory hierarchy, the execution model and the profiling habits that interviewers probe with "this kernel hits 30% of peak bandwidth, why?"

> [AT A GLANCE]
> A GPU is a throughput machine: thousands of threads hide memory latency by always having other work ready. Performance comes from three things in order: **move fewer bytes** (fusion, reuse in SRAM, lower precision), **move them well** (coalesced, vectorized, asynchronous), and **keep the math units fed** (tensor cores, occupancy, no host stalls). Always decide first whether a kernel is memory-bound or compute-bound.

» CPU VS GPU ARCHITECTURE

[TABLE 3.1 // LATENCY MACHINE VS THROUGHPUT MACHINE]

	CPU (server)	GPU (H100 SXM)
cores	tens of large OoO cores	132 SMs, thousands of lanes
threads	~2 per core	up to 2048 resident per SM
latency hiding	caches, branch prediction, OoO	switch warps every cycle
memory	DDR5, ~0.3-0.5 TB/s per socket	HBM3 80 GB at 3.35 TB/s
best at	branchy, serial, low latency	dense, regular, parallel math

» NVIDIA DATA CENTER GPUS: H100, H200, B200 AND GB200

H100 (Hopper) 132 SMs (SXM), 4th-gen tensor cores with FP8, Transformer Engine, Tensor Memory Accelerator (TMA), thread block clusters with distributed shared memory, 80 GB HBM3 at 3.35 TB/s, NVLink 4 at 900 GB/s.

H200 Same Hopper compute with 141 GB HBM3e at 4.8 TB/s. For decode-heavy serving it behaves like a faster, bigger H100: more KV room and about 1.4x the bandwidth.

B200 (Blackwell) Two dies presented as one GPU, 5th-gen tensor cores with FP4 (NVFP4, MXFP4) and FP6, 192 GB HBM3e at 8 TB/s, NVLink 5 at 1.8 TB/s, dense BF16 near 2.25 PFLOPS.

GB200 NVL72 36 Grace CPUs and 72 Blackwell GPUs in one NVLink domain: about 13.5 TB of HBM and 130 TB/s of NVLink switch bandwidth. A 72-GPU scale-up domain changes MoE and disaggregated serving, because expert all-to-all and KV transfer stay on NVLink.

» CUDA PROGRAMMING AND KERNELS

A kernel launches a **grid** of **blocks**; each block runs on one SM; its threads execute in **warps** of 32 that share an instruction stream (SIMT). Blocks cannot synchronize with each other inside a kernel except through cooperative groups or atomics.

[LISTING 3.1 // A COALESCED, WARP-SHUFFLE SUM REDUCTION]

```
__inline__ __device__ float warp_sum(float v) {
  for (int o = 16; o > 0; o >>= 1) v += __shfl_down_sync(0xffffffff, v, o);
  return v;
}
__global__ void reduce_sum(const float* __restrict__ x, float* out, int n) {
  float acc = 0.f;
  for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < n;
       i += gridDim.x * blockDim.x)            // grid-stride loop: coalesced reads
    acc += x[i];
  acc = warp_sum(acc);                          // 32 -> 1 in registers
  __shared__ float s[32];
  int lane = threadIdx.x & 31, wid = threadIdx.x >> 5;
  if (lane == 0) s[wid] = acc;
  __syncthreads();
  if (wid == 0) {
    acc = (lane < blockDim.x / 32) ? s[lane] : 0.f;
    acc = warp_sum(acc);
    if (lane == 0) atomicAdd(out, acc);          // non-deterministic order!
  }
}
```

The atomicAdd makes the result order-dependent in floating point. When the interviewer asks for a deterministic reduction (a reported PracHub question in Chapter 7), write per-block partials to an array and reduce them in a fixed order in a second pass.

» GPU MEMORY HIERARCHY AND BANDWIDTH

[FIG 3.1 // H100 MEMORY HIERARCHY (ORDER OF MAGNITUDE)]

registers	64K x 32-bit per SM	~0 cycle	private to a thread
shared mem/L1	up to 228 KB per SM	~30 cycles	shared by a block, banked (32)
L2 cache	50 MB	~200 cycles	shared by all SMs
HBM3	80 GB at 3.35 TB/s	~500+ cycles	global memory
host DDR	via PCIe 5 (~64 GB/s per direction) or NVLink-C2C on Grace		

Rules that answer most interview questions: neighbouring threads should read neighbouring addresses (coalescing); use 128-bit vector loads; stage reused tiles in shared memory and avoid bank conflicts (pad by one column for a transpose); keep hot values in registers; on Hopper use TMA for asynchronous bulk copies so loads overlap math.

» TENSOR CORES AND PARALLEL EXECUTION

Tensor cores execute small matrix multiply-accumulates (warp-level mma on Ampere, warpgroup-level wgmma on Hopper, tcgen05 with tensor memory on Blackwell). They deliver an order of magnitude more FLOPs than CUDA cores, but only if operands arrive in the right layout and precision fast enough. **Occupancy**, the ratio of resident warps to the maximum, is limited by registers per thread, shared memory per block and threads per block. Higher occupancy hides latency, but a well-pipelined GEMM can run fast at low occupancy because it hides latency with asynchronous copies instead of more warps.

» TRITON PROGRAMMING

OpenAI Triton lets you write block-level kernels in Python; the compiler handles coalescing, shared memory and tensor-core instruction selection. It is the fastest way to a fused kernel that is good enough, and torch.compile (Inductor) emits Triton.

[LISTING 3.2 // FUSED ROW SOFTMAX IN TRITON]

```
@triton.jit
def softmax_kernel(out_ptr, in_ptr, stride, n_cols, BLOCK: tl.constexpr):
    row = tl.program_id(0)
    offs = tl.arange(0, BLOCK)
    mask = offs < n_cols
    x = tl.load(in_ptr + row * stride + offs, mask=mask, other=-float('inf'))
    x = x - tl.max(x, axis=0)                # numerical stability
    num = tl.exp(x)
    y = num / tl.sum(num, axis=0)
    tl.store(out_ptr + row * stride + offs, y, mask=mask)
# one read and one write per element instead of three passes over HBM
```

Newer DSLs push the same idea closer to the hardware: CUTLASS and CuTe for templated C++ GEMMs, CuTe DSL in Python (Perplexity's posting lists it), ThunderKittens and TileLang in research.

» FLASHATTENTION AND KERNEL FUSION

Naive attention writes the $N \times N$ score matrix to HBM, reads it for softmax, writes it again and reads it for the PV product. FlashAttention tiles Q, K and V into SRAM and uses an **online softmax**: for each new block it updates a running max m and running sum ℓ and rescales the partial output, so the full matrix is never materialized:

$$m' = \max(m, \max_j s_j), \quad \ell' = e^{m-m'}\ell + \sum_j e^{s_j - m'}, \quad o' = e^{m-m'}o + \sum_j e^{s_j-m'} v_j$$

HBM traffic drops from $O(N^2)$ to $O(Nd)$ per head, and memory falls from quadratic to linear. FlashAttention-2 improved work partitioning; FlashAttention-3 uses Hopper's asynchrony (warp specialization, TMA, FP8). For decode, where the query is one token, split-K variants (Flash-Decoding) parallelize across the KV length so long contexts still fill the GPU. Fusion in general (bias plus activation, residual plus norm, dequant plus GEMM, RoPE plus KV write) removes round trips to HBM.

» NSIGHT SYSTEMS AND NSIGHT COMPUTE

Nsight Systems The timeline. Shows CPU threads, CUDA API calls, kernel launches, memcpys, NCCL and gaps. Use it first to answer "is the GPU idle?" Typical findings: launch gaps from Python overhead, synchronous copies, a scheduler step that blocks the GPU.

Nsight Compute The microscope for one kernel. Speed of Light section (percent of peak compute and memory), roofline chart, memory workload analysis (L1/L2 hit rates, sectors per request), warp state statistics (stall reasons), source view mapped to SASS.

Also PyTorch profiler for operator-level views, nvidia-smi dmon and DCGM for fleet metrics. Note that "GPU utilization" in nvidia-smi only means a kernel was running, not that the SMs were busy.

» COMPUTE-BOUND VS MEMORY-BOUND WORKLOADS

[FIG 3.2 // ROOFLINE FOR H100 BF16]



```
        |             _/
        |  decode    *   (batch 8: ~8 FLOP per weight byte)
        |__________/_____________________________________ FLOP/byte
                ~295 = ridge point
```

Below the ridge, speedups come from fewer bytes (quantization, fusion, larger batch). Above it, they come from better math (tensor cores, lower precision math, less wasted work).

» PROFILING AND PERFORMANCE OPTIMIZATION WORKFLOW

[1] Define the metric and the workload (tokens per second at a TPOT SLO, a fixed prompt and output length mix).

[2] Timeline first (Nsight Systems): find idle gaps, host overhead and synchronization.

[3] Rank kernels by total time. Fix the top one or two only.

[4] For each, compute the speed of light: bytes needed divided by bandwidth, FLOPs divided by peak. Compare with the measured time.

[5] Change one thing, rerun the same benchmark, keep a log. Check numerical parity after every change.

> **[NUMBERS TO KNOW]**
>
> Warp of 32 threads. H100: 132 SMs, up to 228 KB shared memory per SM, 50 MB L2. Ridge point about 295 FLOP per byte in BF16, about 590 in FP8. FlashAttention cuts attention HBM traffic from quadratic to linear in sequence length.

[SECTION 04 / 11]

LLM INFERENCE OPTIMIZATION AND MODEL SERVING

[FROM ONE GPU TO A GOOD SERVICE]

Engines, batching, memory, caching, speculation, precision and parallelism, with the metrics that tell you whether any of it worked.

> [AT A GLANCE]
> The serving engine decides which tokens run on the GPU each step. Its two jobs are to **keep the batch large** (for throughput) and to **keep each step short** (for latency). Continuous batching, paged KV memory and chunked prefill make the batch large without starving anyone. Prefix caching, speculative decoding and quantization make each token cheaper. Parallelism and disaggregation let one model span many GPUs.

» VLLM, SGLANG, TENSORRT-LLM AND NVIDIA TRITON

[TABLE 4.1 // THE ENGINES INTERVIEWERS NAME]

ENGINE	CORE IDEA	WHERE IT SHINES
vLLM	PagedAttention, continuous batching, V1 engine, prefix caching, many backends	broad model support, OSS default, fast new-model bring-up
SGLang	RadixAttention prefix tree, fast scheduler, structured outputs, PD mode	prefix-heavy and agentic workloads, large MoE (DeepSeek-style) serving
TensorRT-LLM	compiled engines, in-flight batching, fused kernels, PyTorch backend	peak NVIDIA performance, FP8/FP4, tight latency SLOs
Triton Inf. Server	model server: HTTP/gRPC, dynamic batching, backends incl. TRT-LLM	multi-model, ensembles, non-LLM models next to LLMs
Dynamo	NVIDIA distributed serving layer: KV-aware routing, KV transfer (NIXL)	disaggregated prefill/decode, multi-node fleets

Job postings ask for these by name: Baseten lists TensorRT, TensorRT-LLM, vLLM and SGLang; xAI leads SGLang development; Perplexity’s London stack is TensorRT-LLM on Kubernetes; Microsoft mentions Triton and TensorRT-LLM; Google DeepMind’s staff role names vLLM, TensorRT-LLM, SGLang and Dynamo.

» CONTINUOUS BATCHING AND DYNAMIC BATCHING

Dynamic batching (Triton Inference Server style) waits up to a small window to gather requests, runs them as one batch and returns all results together. It suits fixed-cost models (embeddings, classifiers). For generation it wastes slots: a batch finishes only when its longest sequence does.

Continuous (in-flight, iteration-level) batching schedules at every decode step. Finished sequences leave immediately and waiting requests join at the next step. Throughput gains over static batching are large for workloads with varied output lengths, because no slot idles waiting for the longest member.

[FIG 4.1 // STATIC VS CONTINUOUS BATCHING (4 SLOTS)]

```
STATIC      step: 1 2 3 4 5 6 7 8        CONTINUOUS  step: 1 2 3 4 5 6 7 8
slot A          A A A A A A A A          slot A          A A A A A A A A
slot B          B B . . . . . .  idle    slot B          B B E E E F F F
slot C          C C C . . . . .  idle    slot C          C C C G G G G .
```